NAU NORTHERN ARIZONA UNIVERSITY

School of Informatics, Computing, and Cyber Systems

CS 312 - Project Proposal

Group Details:

Student 1 Name	Tyler Vandermooren
Student 1 Email	Tjv55@nau.edu

Project Details:

Title	**Employee Database Management System (EDMS)**
Problem Statement	Organizations require a secure system to manage employee data, with distinct role-based access for admins and employees. Admins should have full control over all employee records, while employees can only view and edit their own profiles.
Project Idea	Develop a comprehensive **Employee Database Management System** using the **PERN stack (PostgreSQL, Express, React, Node.js)**. The system will feature user authentication, role-based access control (admin and employee), and CRUD operations for managing employee records.

Project Components	• **Authentication and Role-Based Access:**
	  ◦ Admins log in to manage all employee records.
	  ◦ Employees log in to view and edit their profiles.
	• **User Interface (React Frontend):**
	  ◦ **Dashboard:** Displays key statistics for both admins and employees.
	  ◦ **Employee List (Admin View):** Admins view all employee records in a table format.
	  ◦ **Employee Profile (Employee View):** Employees can edit their personal information.
	  ◦ **Add Employee Form (Admin Only):** Admins can add new employees with mandatory fields.
	• **Manage Employee Records:**
	  ◦ **Edit Record:** Admins can edit any employee's details; employees can edit only their own record.
	  ◦ **Delete Record:** Only admins can delete records, and this action requires a confirmation prompt.
	• **Search and Filter Functionality:**

	○ **Search Bar:** Admins can search by name or ID; employees search within their profile. ○ **Sort Functionality:** Admins can sort records by various criteria. • **Local Storage Integration and Data Persistence:** ○ PostgreSQL stores employee records; local storage tracks user sessions for role-based functionality.

Task Planning:

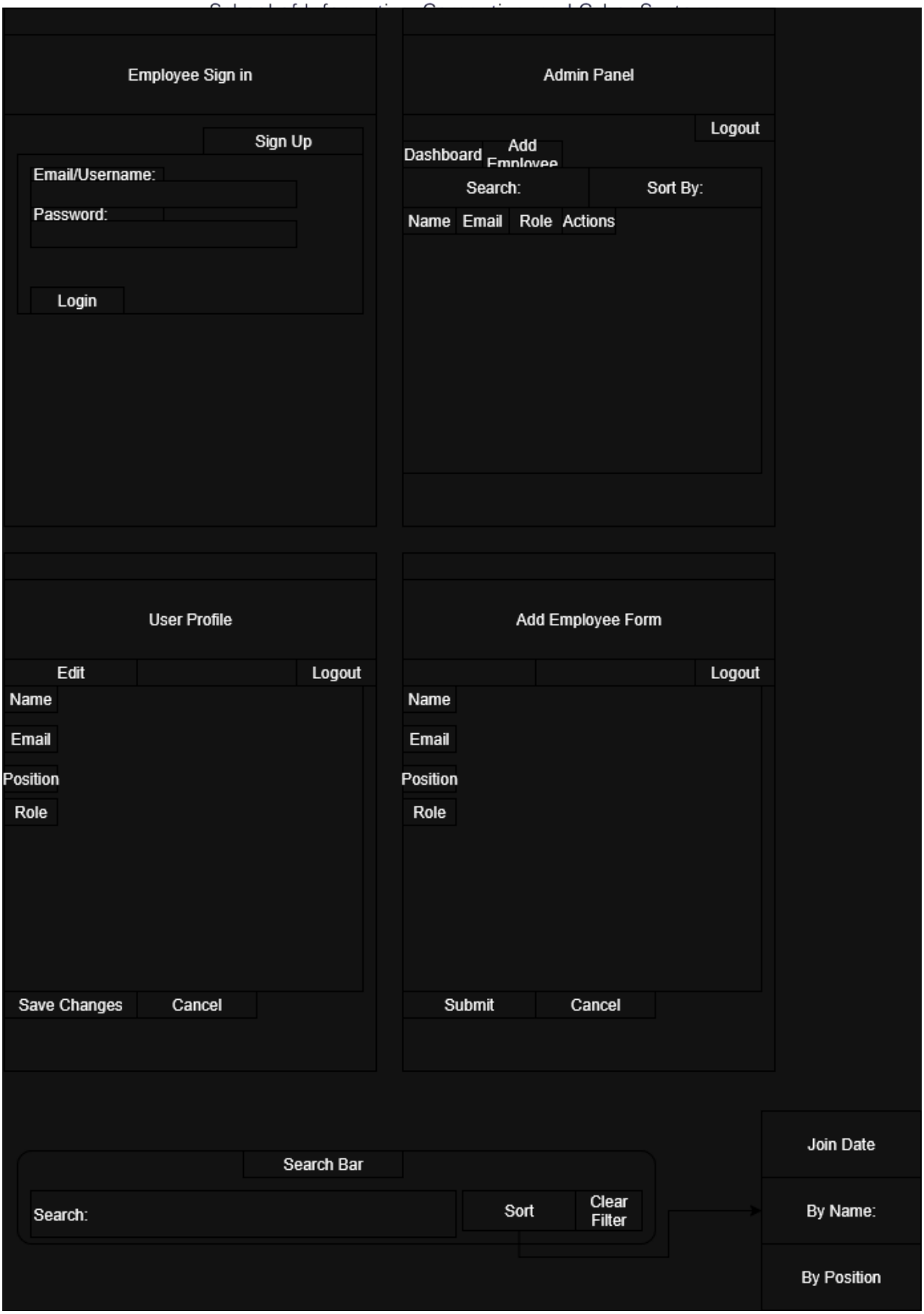
1. Phase-1:

Phase-1 Deliverables	• Set up GitHub Repo and Development Environment (Node, PostgreSQL). • Create PostgreSQL schema for employee records. • Implement user authentication routes (sign up / sign in). • Set up Express backend and test routes with Postman. • Develop React components: Login, Signup, Dashboard skeletons.
Student 1 Tasks	
Initialize GitHub repository and configure local development environment • Create a new private GitHub repository named edms-project. • Clone it locally and set up project structure with /client (React) and /server (Node/Express). • Install required dependencies	
Design and create PostgreSQL database schema for employee records • Define employees table with fields: id, first_name, last_name, email, position, role, created_at, updated_at. • Use pgAdmin to create the database. • Include a users table for authentication with hashed passwords and role flags.	
Implement authentication API: Sign Up and Sign In endpoints • Create Express routes: POST /signup and POST /signin. • Use bcrypt to hash passwords before saving to DB and express-session to track logins. • On successful login, store user info and role in the session.	
Develop React components: Login and Signup forms • Build controlled form components using useState and fetch/axios to POST to backend. • Handle success and error (display error messages).	
Create base Dashboard component (Admin and Employee shell) • Implement a placeholder dashboard that conditionally renders content based on user role. • redirect if not authenticated.	

2. Phase-2:

Phase-2 Deliverables	• Implement role-based access logic. • Complete CRUD functionality (frontend + backend). • Build employee list and profile views. • Add search, filter, and sort functionality for admins. • Finalize styling, test UX, and write project documentation.

Student 1 Tasks
Implement role-based middleware and frontend access control • Write Express middleware to check for admin vs employee role before accessing routes like POST /add, DELETE /employee/:id. • In React, restrict admin-only features from being shown to regular users.
Create and test full CRUD routes for employee records • Backend: Implement routes like GET /employees, POST /employee, PUT /employee/:id, DELETE /employee/:id. • Frontend: Build forms/components to call these routes based on user role.
Develop Admin dashboard with full employee table • Display employee records in a table. • Include Edit and Delete buttons with confirmation prompts. • Fetch employee data from backend and handle updates live.
Build Employee Profile View/Edit page • For employees, display their profile data on login. • Provide a form to update their name, email, position, etc. • Prevent access to any other user's data from the frontend.
Implement search, filter, and sort functionality for employee records (admin only) • Add a search input to filter employees by name, ID, or position. • Implement sorting by name, join date, or position using dropdowns or column headers.
Finalize frontend styling and write project documentation • Apply consistent CSS or Bootstrap styling for professional UI. • Write README.md including project setup, features, technologies, and usage instructions.


Employee Sign in
Sign Up
Email/Username:
Password:
Login
Admin Panel
Logout
Dashboard
Add Employee
Search:
Sort By:
Name
Email
Role
Actions
User Profile
Edit
Logout
Name
Email
Position
Role
Save Changes
Cancel
Add Employee Form
Logout
Name
Email
Position
Role
Submit
Cancel
Search Bar
Search:
Sort
Clear Filter
Join Date
By Name:
By Position